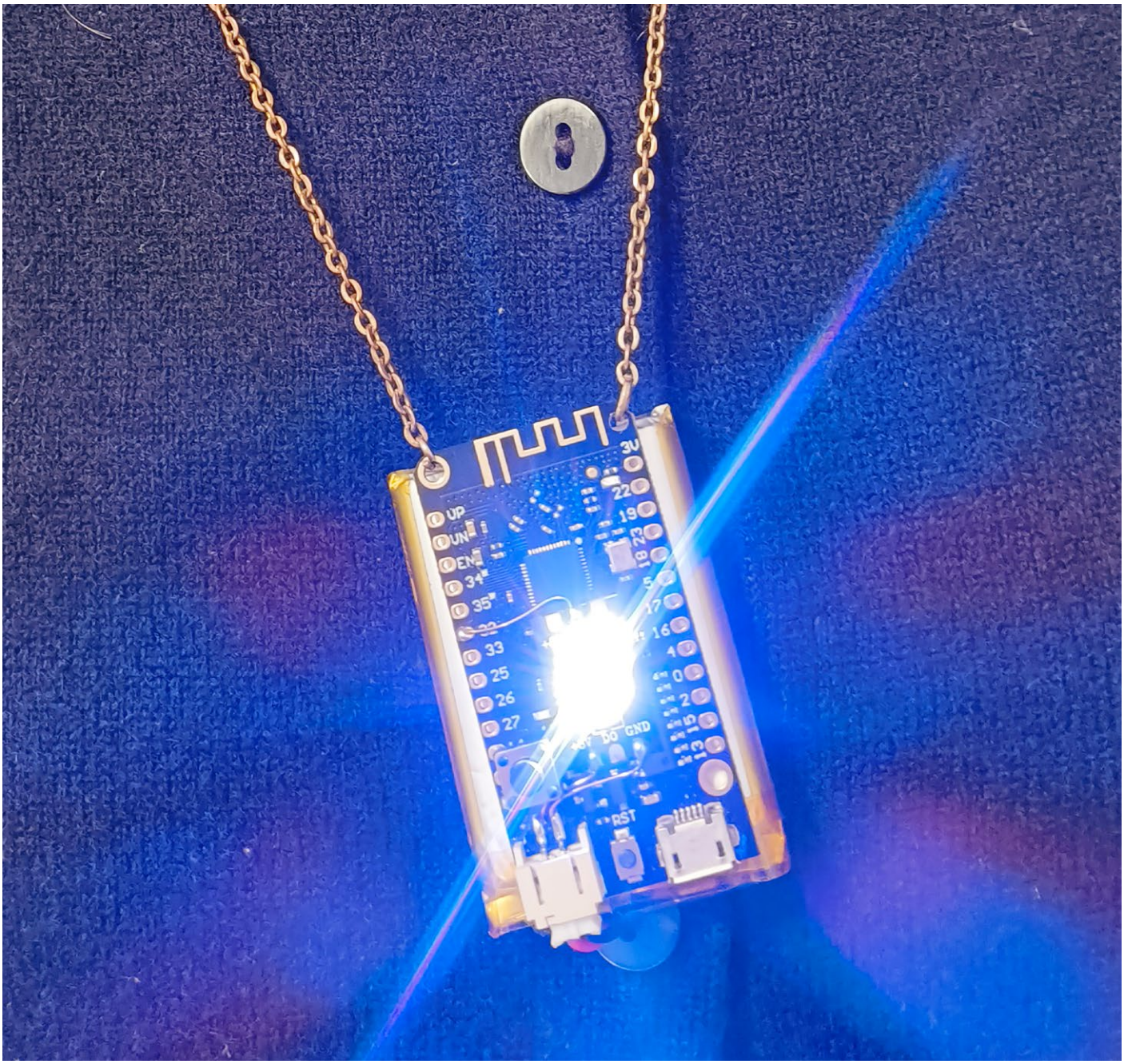
# Flirt-Amulette mit ESP-NOW

**Auf den ersten Blick wirkt es wie ein hübsches Schmuckstück mit LEDs, die sanft pulsieren. Doch bei Annäherung eines ähnlichen Stücks leuchten beide Amulette plötzlich in bunten Farben auf. In diesem Artikel zeige ich, wie Amulette oder Armbänder, die auf ESP-Modulen basieren, über das ESP-NOW-Protokoll kommunizieren können, um z. B. auf Festivals oder Großveranstaltungen ihren Trägern spielerisch zu helfen, sich gegenseitig zu finden.**

von Dr. Stefan Recksiegel






© Copyright by Maker Media GmbH.
Persönliches PDF für alle Leser der Make aus 30625 Hannover

Ich zeige in diesem Artikel zunächst, wie die Kommunikation mithilfe von ESP-NOW und WifiEspNow funktioniert, und bastele dann „Flirt-Amulette", die mit RGB-LEDs anzeigen, ob (und welche) anderen Amulette sich in der Nähe befinden. Die Amulette sind auch in der Lage, einfache Nachrichten auszutauschen, indem man einen auf die Amulette gelöteten Taster drückt.

## Kurzinfo

» **Tragbare Schmuckstücke basteln, die miteinander kommunizieren können**
» **Kommunikation über ESP-NOW und WifiEspNow**
» **Mit Neopixeln und einem Taster farbcodierte Nachrichten verschicken**

### Checkliste

**Zeitaufwand:**
3 Stunden

**Kosten:**
15 bis 20 Euro für zwei Amulette

### Material

» **Wemos Lolin D32, ESP D1 mini, ESP32-C3 Super mini oder Ähnliche**
» **WS2812(b)-LEDs**
» **Akku mit PH-2-Stecker**
» **Taster**

### Werkzeug

» **Lötkolben**

### Mehr zum Thema

» Thomas Fischer, ESP Now: Datenübertragung per Funk, Make 3/22, S. 64
» Carsten Wartmann, Disco is back – Lichtshow mit WLED, Make 2/23, S. 8
» Marcus Hansson, Nowhere-Festival: Das Maker-Paradies in der Wüste, Make 5/24, S. 64

Alles zum Artikel im Web unter *make-magazin.de/xyj8*

## ESP-NOW

Die Module der ESP-Serie von Espressif sind bei Makern sehr beliebt, weil sie sich leicht mit der Arduino IDE programmieren lassen. Im Gegensatz zu den Arduinos verfügen die Chips der ESP8266- und ESP32-Familien über WLAN, aber nicht allen Makern ist bekannt, dass es außer den „normalen" Client- und Access-Point-Modi auch ein eigenes Protokoll für die Ad-hoc-Kommunikation zwischen ESP-Modulen gibt: ESP-NOW (Link in der Kurzinfo).

ESP-NOW kombiniert verschiedene Schichten des OSI-Modells (Open Systems Interconnection), einem Framework zur Strukturierung von Netzwerkkommunikation in mehreren Ebenen, zu einer einzigen Protokollebene. Dies macht es zwar schwierig, ESP-NOW außerhalb des ESP-Ökosystems zu verwenden, aber Kommunikation zwischen ESPs ist einfach aufzusetzen und verbraucht wenig Strom – gut für den mobilen Einsatz mit Batterien oder Akkus.

In der ursprünglichen Version des ESP-NOW, die in der Arduino IDE vorinstalliert ist, muss man die MAC-Adresse (also die Seriennummer des WLAN-Moduls) des Kommunikationspartners im Voraus kennen, was für viele Anwendungen nicht praktikabel ist. Junxiao Shi hat hierfür die auf ESP-NOW aufbauende Bibliothek WifiEspNow geschrieben (Link in der Kurzinfo), mit der ESP-Module in Reichweite sich gegenseitig erkennen können. Die Bibliothek kann wie üblich über den Library Manager in der Arduino IDE installiert werden.

## Geeignete ESPs

Mein Code funktioniert auf allen Chips der ESP8266/8285- und der ESP32-Serie, aber die Verdrahtung der internen LEDs und der (falls vorhandenen) eingebauten Taster hängt vom Modul ab. Wenn ihr nicht im Datenblatt oder auf Schaltplänen nachschauen wollt, empfehle ich euch entweder einen ESP D1 mini, bei dem die eingebaute LED zwischen GPIO2 des ESP8266 und 3,3 V hängt (leider ohne eingebauten Taster) oder besser noch einen ESP32-C3 Super Mini, der eine LED auf GPIO8 und einen Taster auf GPIO9 hat. Beide Module gibt es für unter 5 Euro pro Stück auf den bekannten Verkaufsplattformen zu kaufen. Ihr braucht mindestens zwei Module, es können aber ruhig verschiedene sein.

## Die Sketches

Ich habe drei verschiedene Sketches geschrieben, die alle über den Link in der Kurzinfo zu finden sind. Im ersten (Make_ESPnow_minimal.ino) werden die MAC-Adressen der ESPs (alle Typen der Serien ESP8266 und ESP32), auf denen der gleiche Sketch läuft, auf das Empfängermodul übertragen und durch unterschiedliche Blinkrhythmen der eingebauten LEDs angezeigt.

Ein etwas erweiterter Sketch (Make_ESPnow_button.ino) ermöglicht das Senden von Nachrichten zwischen den ESPs: Wenn ein eingebauter oder aufgelöteter Taster auf einem der ESPs gedrückt wird, wird auf den anderen ESPs eine Funktion aufgerufen, die eine serielle Meldung ausgibt und den Empfang per LED anzeigt.

Der dritte Sketch (Make_ESPnow.ino) steuert unter Verwendung der WS2812FX-Bibliothek LEDs an. Wenn andere Module erkannt werden, dann pulsiert die LED langsam in einer Farbe, die von den letzten drei Bytes der MAC-Adresse des Partners abhängt. Wenn man schon einmal in Reichweite dieses Moduls war, kann man also eventuell die Farbe wiedererkennen. Die RGB-Werte werden aus den drei letzten Bytes der MAC-Adresse ermittelt. Im Gegensatz zu den ersten Bytes, die den Hersteller des Moduls angeben und daher oft gleich sind, laufen diese als Seriennummer über den gesamten Bereich von 0 bis 255. Das führt mit großer Wahrscheinlichkeit dazu, dass die verwendeten Module auch Farben zeigen, die gut voneinander zu unterscheiden sind.

Wenn mehrere andere Module in Reichweite sind, wechselt die pulsierende Farbe auf dem eigenen Modul zwischen den zu diesen Modulen gehörigen. Auch hier kann man durch Drücken eines Tasters die LEDs auf den anderen Modulen für ein paar Sekunden hell aufblitzen lassen.

Die drei Sketches sind untereinander kompatibel – es können also unterschiedliche Sketches auf verschiedene Amulette aufgespielt werden, natürlich mit entsprechender Einschränkung der Funktionalität. Die fest verbauten LEDs auf den verschiedenen Modulen können beispielsweise nur einfarbiges Licht (meist blau) abgeben.

## Wie die Module sich finden

Schauen wir uns jetzt den ersten und einfachsten der drei Sketches näher an (Listing 1). Er gibt den Modulen die Anweisung, per LED anzuzeigen, ob ein anderes Modul in Reichweite ist. Wenn das der Fall ist, wird die LED lang angeschaltet und kurz ausgeschaltet. Wenn kein anderes Modul erkannt wird, ist es genau andersherum: kurz ein, lang aus. Außerdem wird eine Information über gefundene Partner über die serielle Schnittstelle ausgegeben, was für Debugging-Zwecke sinnvoll ist. Wenn ihr einen ESP32-C3 nutzt, muss unter

© Copyright by Maker Media GmbH.
Persönliches PDF für alle Leser der Make aus 30625 Hannover

## Listing 1

```
#define STATUS_LED 8
#define LEDON LOW
#define LEDOFF HIGH

#if defined(ARDUINO_ARCH_ESP8266)
 #include <ESP8266WiFi.h>
#elif defined(ARDUINO_ARCH_ESP32)
 #include <WiFi.h>
#endif

#include <WifiEspNowBroadcast.h>
#define WIFI_CHANNEL 1
#define MAX_PEERS 20

void setup() {
  pinMode(STATUS_LED, OUTPUT);

  Serial.begin(115200); Serial.println();
  Serial.printf("Starting ESPnow demo, LED on GPIO%i\n", STATUS_LED);
  WiFi.persistent(false);
  bool wifiok = WifiEspNowBroadcast.begin("ESPNOW", 3);
  if (!wifiok) { Serial.println("WifiEspNowBroadcast.begin() failed");
    ESP.restart(); }
  Serial.print("MAC address of this node is ");
  Serial.println(WiFi.softAPmacAddress());
}

void loop() {
  WifiEspNowBroadcast.loop();
  WifiEspNowPeerInfo peers[MAX_PEERS];
  int nPeers = WifiEspNow.listPeers(peers, MAX_PEERS);
  Serial.printf("Number of peers: %i ",nPeers);
  for (int i = 0; i < nPeers; ++i) {
    Serial.printf(" %02X:%02X:%02X:%02X:%02X:%02X ", peers[i].mac[0],
      peers[i].mac[1], peers[i].mac[2], peers[i].mac[3],
      peers[i].mac[4], peers[i].mac[5]); }
  Serial.println();

  // if connected: short off, long on - if not: short on, long off
  if( nPeers>0 ) { digitalWrite(STATUS_LED, LEDOFF); delay(50);
    digitalWrite(STATUS_LED, LEDON);}
  else           { digitalWrite(STATUS_LED, LEDON); delay(50);
    digitalWrite(STATUS_LED, LEDOFF);}
  delay(950);
}
```

Tools in der Arduino IDE „USB CDC On Boot" aktiviert sein, damit die serielle Schnittstelle aktiv ist!

Bei der Initialisierung wird `WifiEspNowBroadcast` gestartet. Bei jedem Durchlauf des Loops wird die Kontrolle an `WifiEspNowBroadcast` übergeben, damit andere Module in Reichweite gefunden werden können. Die MAC-Adressen der gefundenen Module werden ausgegeben und der Blinkrhythmus der internen LED wird entsprechend gewählt.

## Daten versenden

Wirklich nützlich wird ESP-NOW, wenn wir das Protokoll nicht nur dafür verwenden, andere Module zu finden, sondern auch, um ihnen Daten zu schicken. Im Rahmen der Längenbeschränkung von 250 Bytes pro Paket sind beliebige Daten möglich. Wir senden nur die kurze Zeichenkette „Button pressed!". Den kompletten Sketch findet ihr unter dem angegebenen Link. Der entscheidende Befehl zum Senden ist

```
WifiEspNowBroadcast.send(
  reinterpret_cast<const uint8_t*>
  (message), strlen(message)+1);
```

Dieser Befehl wird ausgeführt, wenn der (eingebaute oder selbst angelötete) Taster gedrückt wird. Wer nicht sehr versiert in C ist, tut sich wahrscheinlich mit diesem Befehl etwas schwer: Vor dem Senden wird die Zeichenkette (String) als (konstante) vorzeichenlose 8-Bit-Ganzzahlen (uint8) reinterpretiert. Es wird nichts umgewandelt, die Daten liegen genauso im Speicher wie bisher, es ändert sich lediglich die Bedeutung, denn `WifiEspNowBroadcast.send` erwartet `uint8` und keine Zeichen (`char`). Warum ist die Länge der zu sendenden Daten um eins länger als der String selbst? Strings werden durch einen 0-Char beendet, der nicht zur Nettolänge zählt. Dieses Endzeichen wollen wir aber mitsenden, damit der Empfänger weiß, dass es sich um einen String handelt. Sonst sind es nur Rohdaten.

## Daten empfangen

Die Sache mit dem Empfang ist etwas komplizierter: Wir wollen ja nicht die ganze Zeit darauf warten, ob jemand anderes vielleicht gerade Daten schickt. Daher registriert man mit

```
WifiEspNowBroadcast.onReceive(
                processRx, nullptr);
```

eine sogenannte Callback-Funktion (hat nichts mit Casting-Shows zu tun, auch wenn wir oben einen Pointer casten mussten). Diese Funktion wird aufgerufen, sobald `WifiEspNowBroadcast` ein Paket erhalten hat. Wie die Funktion selbst aussieht, seht ihr in Listing 2.

Die genauen Argumente der Funktion müssen uns hier gar nicht so sehr interessieren. Wichtig ist nur, dass in der Variablen `buffer[]` die empfangenen Daten übergeben werden, und dass wir mit `mac[]` auf die Adresse des Senders zugreifen können.

## Listing 2

```
void processRx(const uint8_t mac[WIFIESPNOW_ALEN], const uint8_t* buffer, size_t count, void* arg){
  Serial.printf("Message from %02X:%02X:%02X:%02X:%02X:%02X: ", mac[0], mac[1], mac[2], mac[3], mac[4], mac[5]);
  for (size_t i = 0; i < count; ++i) Serial.printf(" %i",buffer[i]); Serial.println();
  if(buffer[count-1]==0) Serial.printf("Message is a string: '%s'\n", (char*)buffer);
  if(strcmp(message, (char*)buffer) ==0 ) lastReceivedMillis = millis();
}
```



© Copyright by Maker Media GmbH.
Persönliches PDF für alle Leser der Make aus 30625 Hannover

Wenn das letzte Byte `0` ist, handelt es sich wohl um einen String, den wir dann ausgeben. Wenn der String unserem „Button pressed!" entspricht, speichern wir die augenblickliche Systemzeit in der Variable `lastReceivedMillis`, um dann z. B. die eingebaute LED aufleuchten zu lassen.

## WS2812-LEDs ansteuern

Damit kommen wir dann endlich zum Fun-Part! Ich habe diese Technik bereits 2018 benutzt, um Beleuchtungen auf zwei Fahrrädern auf dem Burning-Man-Festival in Nevada zu steuern (Bild 1). Die synchronen Regenbögen und Blinkeffekte auf den nebeneinander fahrenden Rädern sahen in der dunklen Wüste wirklich spektakulär aus, auch weil sie deutlich heller waren als die Beleuchtung der zahllosen anderen Räder dort. Wir sind oft auf unsere Räder angesprochen und gefragt worden, wie das funktioniert. Ich war sehr stolz darauf, bei meinem ersten Besuch dort gleich etwas mitgebracht zu haben, was so viel Aufmerksamkeit erregt hat.

Wir beschränken uns hier mal auf kleine Anhänger, aber der Zahl der LEDs, die wir ansteuern können, ist im Wesentlichen nur durch den Stromverbrauch eine Grenze gesetzt.

Wir verwenden die WS2812FX-Bibliothek, die sich wiederum auf Adafruits Neopixel-Bibliothek stützt (siehe Link in der Kurzinfo für die vollständige Dokumentation der Bibliothek). Wenn ihr mal zum Beispiel auf dem Nowhere-Festival, worüber neulich hier in der Make berichtet wurde (siehe Make 5/24, S. 64), LED-Effekte seht, achtet darauf: In sehr vielen Fällen sind diese Effekte aus dem Repertoire von WS2812FX. Anders als der Name suggeriert, kann diese Bibliothek auch viele andere LED-Typen ansteuern. Wir verwenden aber tatsächlich die namensgebenden WS2812(b)-LEDs, die es sehr günstig auf vorgefertigten Streifen zu kaufen gibt.

Die Bibliothek wird verwendet, indem man im `setup()` ein WS2812FX-Objekt definiert, initialisiert und startet (Listing 3). Zudem müssen ein Effekt, eine Geschwindigkeit und die Helligkeit ausgewählt und gegebenenfalls eine Grundfarbe festgelegt werden.

Wichtig ist, dass im Loop die Funktion `ws2812fx.service();` aufgerufen wird. Diese „worker function" übernimmt die Aktualisierung der Farben und Helligkeiten gemäß dem gewählten Effekt und die Übertragung der Daten an die LEDs. Die WS2812-LEDs hängen in einer Kette: Die erste LED erhält die Daten für alle LEDs, setzt ihre Rot-, Grün- und Blauwerte nach den ersten drei Bytes und reicht den Rest der Daten über ihren Ausgang an die folgenden LEDs durch. Die zweite LED verwendet dann die ersten drei bei ihr ankommenden Bytes und so fort. Die LEDs haben also keine explizite Adresse, vielmehr ist ihre

Foto: Stefan Recksiegel

**Bild 1: Hier wird die gleiche Technik benutzt, um auf zwei nebeneinander fahrenden Fahrrädern synchrone Regenbögen zu erzeugen.**

### Listing 3

```
WS2812FX ws2812fx = WS2812FX(NUM_LEDS, WS2812_PIN, NEO_GRB);
ws2812fx.init();
ws2812fx.setMode(12);
ws2812fx.setSpeed(1000);
ws2812fx.setBrightness(200);
ws2812fx.setColor(0xFF0000);
ws2812fx.start();
```

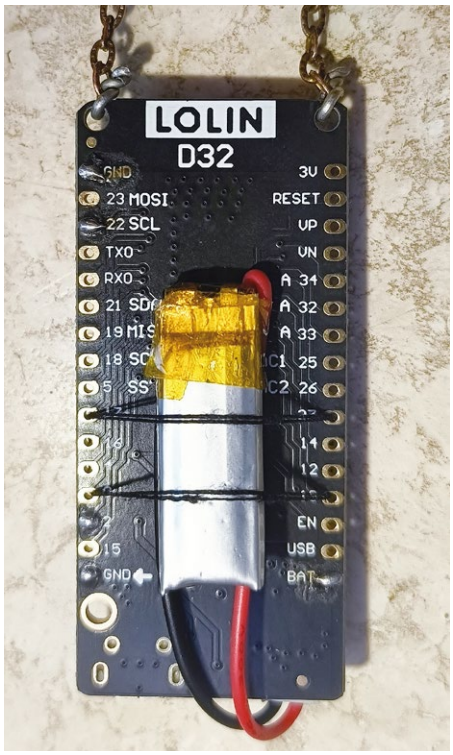



**Bild 2: Der hier verwendete Li-Ion-Akku stammt aus einem Bluetooth-Adapter für ein etwas älteres Paar Bose-Kopfhörer.**

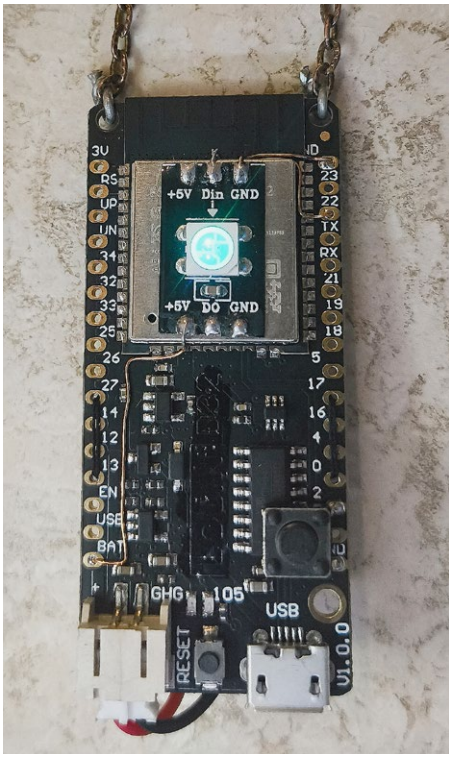



**Bild 3: Hier ist ein originaler Wemos D32 mit dem ESP32 in einem kleinen Blechgehäuse zu sehen. Darauf eine WS2812-LED aus einem Streifen.**

© Copyright by Maker Media GmbH.
Persönliches PDF für alle Leser der Make aus 30625 Hannover

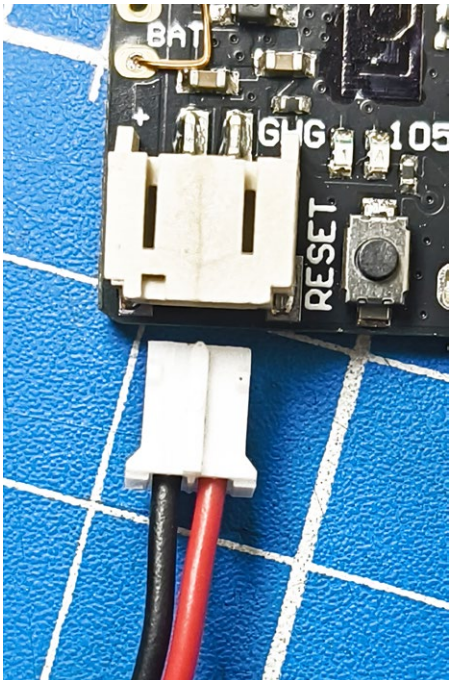



Bild 4: Hier müssen die Kabel des Akkus noch getauscht werden.

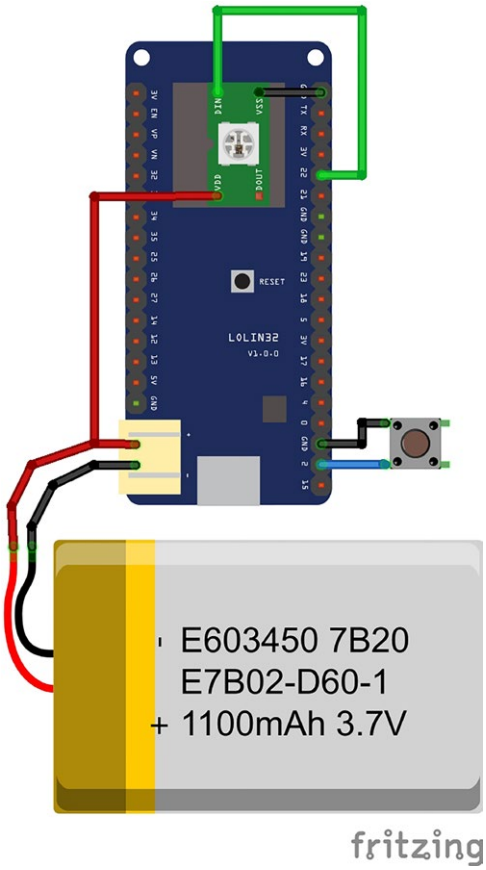



Bild 5: Der Schaltplan ist sehr übersichtlich. Taster und die Datenleitung der LED können an fast alle GPIOs angeschlossen werden.

Adresse durch ihre Position in der Kette vorgegeben und man kann diese LEDs beliebig aneinanderhängen. Sogar Verzweigungen sind möglich – wobei dann natürlich beide Äste die gleichen Effekte zeigen.

Der Pin, an den die LEDs (bzw. die erste LED) angeschlossen werden, ist im Wesentlichen beliebig, aber beim ESP8266 sollte man nicht GPIO0 (wird verwendet, um den ESP beim Booten in den Programmiermodus zu schalten), GPIO1 und GPIO3 (serielle Schnittstelle) oder GPIO2 (interne LED auf vielen Modulen) verwenden. GPIO4 ist eine gute Wahl.

## Das Amulett bauen

Für das Basteln eines umhängbaren Amuletts eignet sich ein Wemos Lolin D32 (oder einer der vielen Nachbauten mit ähnlicher Funktionalität) aus zwei Gründen besonders gut: Zum einen, weil zwei passende Löcher vorhanden sind, an denen eine Schnur befestigt werden kann, um das Amulett um den Hals zu tragen, und zum zweiten, weil hier direkt ein Li-Ion-Akku angeschlossen werden kann, der über USB aufgeladen wird (Bild 2 und 3).

Achtung, die Hersteller von Akkus mit PH-2-Stecker und die Boardhersteller scheinen sich nicht über die Polarität der Stecker einigen zu können! Bei allen von mir beschafften Akkus musste ich die Kontakte vorsichtig aus dem Steckergehäuse lösen (mit einer Stecknadel die Widerhaken herunterdrücken) und tauschen (Bild 4), damit die Polarität mit der Buchse auf dem Modul übereinstimmte. Glücklicherweise waren die Steckverbinder überall eindeutig beschriftet und die Kabel der Akkus farblich gekennzeichnet (+: rot, –: schwarz).

Ich hatte einen 103450-Li-Ion-Akku herumliegen, der mit 2000 mAh mehr als genug Kapazität hat (ca. 10 Stunden bei 200 mA Verbrauch, je nach Leuchteffekten und ESP-Modell). Er lässt sich auch noch gut auf der Rückseite der Module befestigen. Ein kleinerer Akku tut es aber auch. Beispielweise sind in vielen Einweg-E-Zigaretten kleine Li-Ion-Akkus mit 3,7 V Nennspannung verbaut, die sich sehr gut für Bastelprojekte eignen. Allerdings muss man hier noch Lade-/Schutzschaltungen nachrüsten, die bei aufladbaren Geräten schon vorhanden sind.

An dieser Stelle fragt man sich vielleicht, ob ein 3,7-V-Akku für den ESP32 und die WS2812-LEDs geeignet ist. Meine Antwort ist ja: Man kann sowohl den ESP als auch die WS2812 direkt (mit Akku-Schutzschaltung, falls nicht in den Akku eingebaut, natürlich!) an einen Li-Ion-Akku anschließen. Die 3,3 bis 4 V sind dem ESP eigentlich etwas zu viel und dem WS2812 etwas zu wenig, beide funktionierten bei mir aber klaglos.

Was wir noch brauchen, ist mindestens eine WS2812-LED. Eine einzelne ist lichtstark genug, um den gewünschten Effekt zu erzeugen. Zu guter Letzt sollte auch ein Taster auf dem ESP-Modul eingebaut werden, falls dieser nicht eh schon vorhanden ist. Der Reset-Taster zählt nicht!

Der Schaltplan (Bild 5) ist denkbar einfach: Der Taster und die Datenleitung der WS2812-LED können an fast jeden GPIO angeschlossen werden. Die entsprechenden GPIOs müssen natürlich dann korrekt im Sketch eingetragen werden. Um die richtige Stelle zu finden, gibt es am Anfang des Sketches Kommentare mit Hinweisen zur Pinbelegung der verschiedenen hier erwähnten Module.

Wer möchte, kann statt des Wemos Lolin D32 auch den weit verbreiteten ESP8266 D1 Mini nehmen (Bild 6). Dieser hat allerdings keine eingebaute Akku-Ladeschaltung. Der Taster muss dann an GPIO0 und eine WS2812-LED an GPIO4 (auf den Modulen bezeichnet als D2) angelötet werden. Bei dem D1 mini habe ich die Leiterbahn des eingebauten Reset-Tasters aufgetrennt und stattdessen eine Verbindung zu GPIO0 (D3) hergestellt, sodass dieser ehemalige Reset-Taster für unsere Zwecke brauchbar wird. Die WS2812-LEDs auf den Bildern sind teilweise einzelne LEDs von einem längeren Streifen und auf dem D1 mini sitzt ein Adafruit-Modul, das die gleiche Funktionalität bietet.

Wer Akku und LED nicht direkt am ESP-Modul befestigen möchte, ist mit dem ESP32-C3 super mini gut bedient: Dieses Modul ist sehr klein, hat aber trotzdem noch Anschlüsse im relativ einfach zu lötenden 2,54-mm-Raster.

Bei dem D32-Klon (rechts im Bild 6) ist der Taster an GPIO14 angeschlossen (weil der gerade einen zum Taster passenden Abstand zum nächsten GND hatte). Die Datenleitung der LED hängt an GPIO32 (ebenfalls gerade an passender Position) und die interne LED an GPIO22 (weil der wohl für den Modulhersteller gerade passend neben dem Platz für die LED lag).

Wichtig ist, dass die WS2812-LEDs, die ja eigentlich eine 5-V-Versorgungsspannung haben möchten, direkt an die Akku-Anschlüsse auf dem Modul gelötet werden. Mit den 3,3 bis 4 V der Akkus kommen die Konstantstromquellen in den LEDs gut zurecht, aber sie hinter dem Spannungsregler des ESP anzuschließen, ist bei einer LED unnötig und bei vielen eine sehr schlechte Idee. Der Grund dafür ist, dass der kleine Spannungsregler auf dem Mikrocontroller-Board nicht für die hohen Stromlasten ausgelegt ist, die eine größere Anzahl von LEDs erfordern. Außerdem verschwendet die unnötige Regelung wertvolle Batterieenergie. Bei voller Helligkeit auf allen drei Farbkanälen verbraucht eine einzige WS2812-LED bis zu 60 mA.

## Zeit zum Flirten

Und was machen wir jetzt mit der fertigen Hardware? Das ist natürlich völlig euch über-



© Copyright by Maker Media GmbH.
Persönliches PDF für alle Leser der Make aus 30625 Hannover

lassen, aber eine Idee, die ich selber ausprobiert habe, und mit der meine Freunde und ich jede Menge Spaß hatten, war es, beleuchtete Schals mit diesen ESP-Modulen auszustatten (Bild 7). In diesen Schals aus einem Stück Kunstpelz sitzt ein Streifen mit WS2812-LEDs, und wenn man einen Taster drückt, schalten alle gleichzeitig auf einen neuen, vorprogrammierten Modus um. Das sieht sehr beeindruckend aus, wenn mehrere Leute mit solchen Schals auf einer Party sind, insbesondere weil alle Schals synchron leuchten.

Interessant sind unsere Gadgets in Szenarien, in denen die Träger nicht vorher wissen, wer sonst noch so ein Amulett trägt, und somit auf das angezeigte Lichtmuster achten müssen, wenn andere Menschen in ihre Nähe kommen. Dies wird durch das Easter-Egg im letzten Sketch bereichert: Wenn auf einem der Module der Taster gedrückt gehalten wird, dann blitzen die LEDs auf den anderen Modulen in einem sehr schnellen Rhythmus in Rot und Weiß auf. Man kann also andere Träger solcher Amulette auf sich aufmerksam machen. Die Reichweite von der Signalübertragung hängt von der Art der Umgebung ab, aber mit zehn Metern ist auf jeden Fall zu rechnen.

Ob ihr diese Module wirklich zum Flirten verwenden wollt und ob ihr überhaupt mit Menschen, die mit solchen Dingern herumlaufen, flirten wollt, ist euch überlassen, aber hoffentlich habt ihr etwas über Ad-hoc-WLAN-Datenübertragung und coole LED-Gadgets gelernt. Ich bin gespannt auf weitere Anwendungsbeispiele! *—mch*

**Bild 7: Die gleichen ESP-Module können z. B. auch für beleuchtete Schals verwendet werden.**

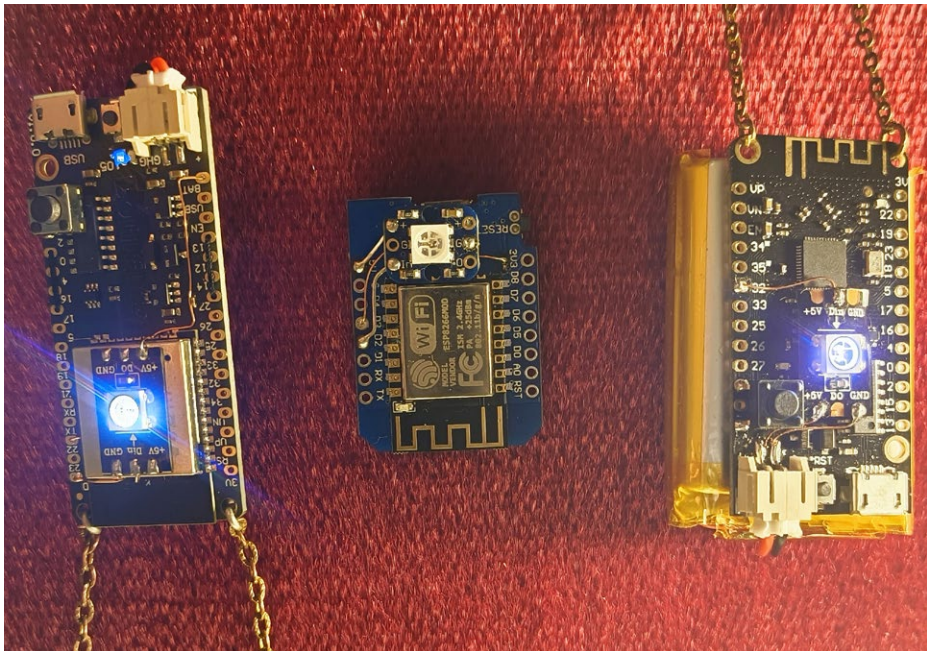


**Bild 6: Links ist ein originaler Wemos D32, in der Mitte ist ein ESP8266 D1 Mini mit Adafruit WS2812-Modul und zum GPIO0-Taster umfunktioniertem Reset-Taster. Rechts sehen wir einen D32-Klon ohne Blechgehäuse. Die dünnen Verbindungen auf den Fotos sind Kupferlackdraht, also isoliert.**

## Wie Konzertarmbänder funktionieren

Die Übertragung von Informationen in tragbare Schmuckstücke kann nicht nur über WLAN erfolgen, wie in unseren Flirt-Amuletten. Drei weitere Möglichkeiten werden in sogenannten Konzertarmbändern genutzt: Infrarotstrahlung, Radiofrequenzen und Bluetooth. Beispielweise erhält jeder Besucher bei den aktuellen „Taylor Swift Eras World Tour"-Konzerten ein LED-bestücktes Armband. In diesen ist ein Infrarotsensor integriert, der in den Stadien von großen Infrarotscheinwerfern Signale empfangen kann, um Lichteffekte und auch Muster auf den Tribünen zu erzeugen. Das funktioniert, indem die beweglichen Scheinwerfer das Infrarotsignal gezielt in bestimmte Bereiche des Publikums senden und so nicht nur alle Armbänder in der gleichen Farbe aufleuchten lassen, sondern auch räumliche Muster erzeugen.

Bereits 2012 hat die britische Band Coldplay mit den sogenannten Xylobands damit begonnen. Die ersten Versionen kommunizierten per Funk, worüber vorprogrammierte Muster aktiviert wurden. Auf der aktuellen Welttour (bis November 2024) verwendet Coldplay Armbänder des Herstellers Pixmob, die zudem komposttierbar sind und teilweise Bewegungssensoren besitzen, um noch weitere Effekte zu erzeugen.

Einen dritten Weg gehen viele K-Pop-Bands aus Korea. Dort werden bei Konzerten sogenannte Light Sticks eingesetzt, für die die Fans viel Geld ausgeben, um das Gefühl haben zu können, dabei zu sein. Die Light Sticks, die während des Konzerts in der Hand gehalten werden, sehen aus wie eine Mischung aus Anime-Zauberstab und Taschenlampe.

Hier wird oft Bluetooth für die Signalübertragung verwendet und die Konzertbesucher werden aufgefordert, eine App zu installieren, damit es funktioniert. Wenn alle gleichzeitig leuchten, entsteht ein sogenannter „Ozean". Sind die Fans aber mit der einen oder anderen Boyband unzufrieden, werden die Light Sticks alle ausgeschaltet, was als „Black Ocean" bezeichnet wird – ein Akt der gewollten Respektlosigkeit.

Die Zeiten, in denen das Publikum Feuerzeuge und Handy-Displays zum Leuchten brachte, sind eindeutig vorbei!

© Copyright by Maker Media GmbH.
Persönliches PDF für alle Leser der Make aus 30625 Hannover